00 01 02 03 04 05 06 07 08 09 0A 0B **0C** 0D

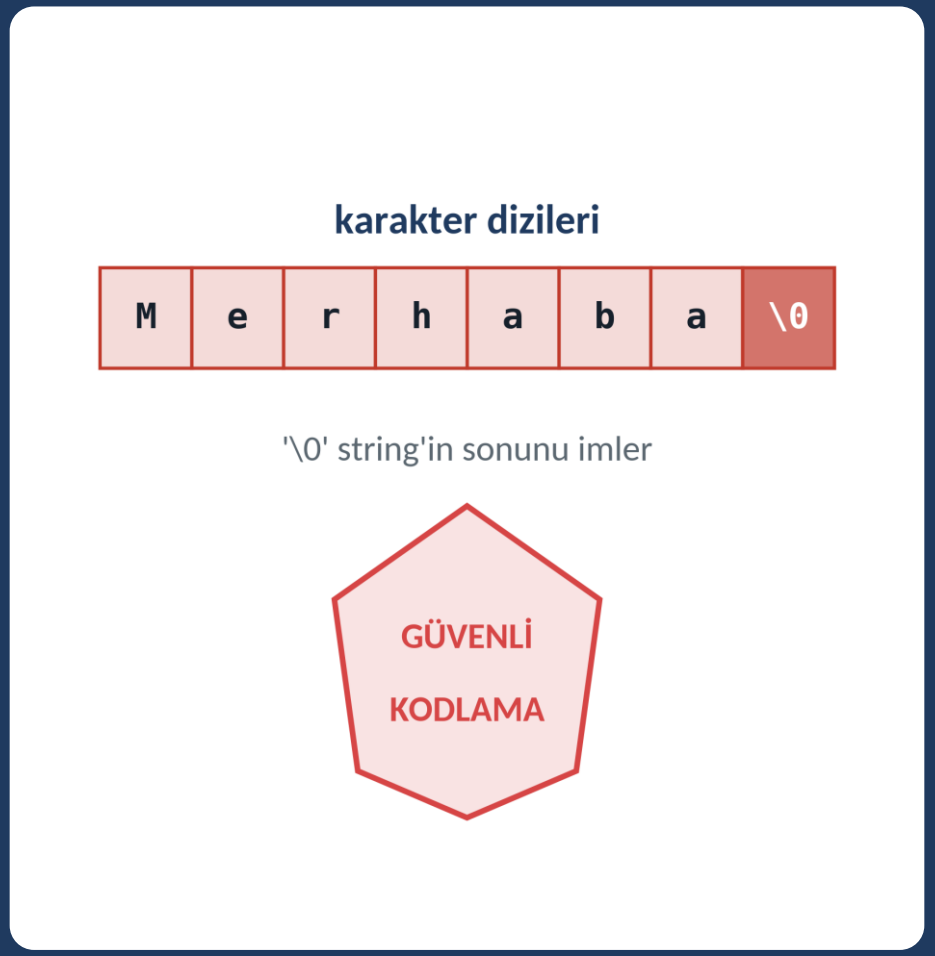
ALGORİTMA VE PROGRAMLAMA – I

HAFTA 13

Karakter Dizileri ve Metin İşleme: Güvenli Kodlamaya Giriş

'\0' ile biten char dizileri · güvenli girdi (fgets, snprintf) · string.h ve ctype.h · ters çevirme, palindrom, Sezar, kelime sayma, frekans · UTF-8 · argc/argv · tampon taşması ve güvenli kodlama

MODÜL VI · METİN, KAYIT VE KALICILIK



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Güz

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Bilgisayar ve Program	**H2** Algoritmik Düşünme	**H3** Değişkenler ve Veri Tipleri	**H4** Operatörler ve İfadeler	**H5** Karar Yapıları	**H6** Döngüler I	**H7** Döngüler II + Ara Proje
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Fonksiyonlar I	**H9** Kapsam, Yığın, Özyineleme	**H10** Diziler I	**H11** Diziler II: Matrisler	**H12** İşaretçiler ve Bellek	**H13** **Karakter Dizileri**	**H14** Yapılar, Dosyalar, Final
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 · Final projesi: H14

GERİ ÇAĞIRMA · H12 · DİZİ–İŞARETÇİ

`int a[] = {10,20,30,40}; int *p = a + 1;` → `a[2], *(a + 2), p[1]` ne yazdırır?

quiz/H13_Q01_dizi_denklik.c · satır 7–13

```c
int main(void)
{
    int a[] = {10, 20, 30, 40};
    int *p = a + 1;                 /* a[1]'i gösterir */
    printf("%d %d %d\n", a[2], *(a + 2), p[1]);
    return 0;
}
```

A `30 30 30`

B `20 30 40`

C `30 30 20`

D `20 20 20`

GERİ ÇAĞIRMA · H12

S2 (sizeof tuzağı) ve S3 (bildirimde *): gerçek çıktılar

S2. `double *d = malloc(4 * sizeof *d);`
`sizeof d` ve `4 * sizeof *d`?

S3. `int *p, q;`
`sizeof p` ve `sizeof q`?

GERÇEK ÇIKTILAR (Q02, Q03)

```
$ ./q02
8 32
$ ./q03
8 4
```

BÜYÜK SORU

Haftanın Büyük Sorusu

1988’de İnternet’e bağlı bilgisayarların kayda değer bir bölümünü etkileyen solucan, hangi C fonksiyonunun zaafından yararlandı?



String = bayt dizisi

Bir ad, parola, dosya satırı: hepsi '\0' ile biten char dizisidir.



Güç: karakter işleme

Metni karakter karakter sayar, çevirir, şifreler, ararız.



Sorumluluk: taşma

'\0' unutulursa ya da tampona sığmayan metin yazılırsa bellek bozulur.

Yanıtı haftanın sonunda gerçek araç çıktılarıyla (ASan, kanarya) kanıtlayacağız.

GERÇEK DÜNYADAN

2 Kasım 1988

Morris solucanı ve bir fonksiyonun bedeli

- `fingerd.c`: `char line[512]; gets(line);` — `gets` **sınır denetlemez**
- Solucan 536 baytlık bir dizi gönderdi → tampon taştı, yığındaki **dönüş adresi** bozuldu
- Sonuç: solucan kendi komutlarını çalıştırdı; VAX/4 BSD'de etkili, Sun'da yalnızca çökme
- Kalıcı iz: `gets` **C11'de standarttan çıkarıldı**; yerine `fgets`

≈ 6.000

etkilenen bilgisayar (24 saatte)

≈ 60.000

o dönemde İnternet'e bağlı makine

1/10

İnternet'in etkilenen oranı

Spafford (1989) · Federal Bureau of Investigation (2018) · 4.3BSD fingerd.c (1985)

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 13.1	C’de karakter dizisinin '\0' ile sonlanan char dizisi olduğunu bellek düzeyinde açıklar.
ÖÇ 13.2	Karakter ve metin girişi/çıkışını güvenli biçimde (fgets, snprintf) yapar.
ÖÇ 13.3	string.h ve ctype.h fonksiyonlarını kullanır; temel string fonksiyonlarını (strlen, strcpy, strcmp) kendisi gerçekler.
ÖÇ 13.4	Karakter düzeyinde metin işleme algoritmaları (sayma, dönüştürme, palindrom, kelime sayma, şifreleme) tasarlar.
ÖÇ 13.5	Tampon taşması açığının mekanizmasını ve önleme yollarını açıklar.

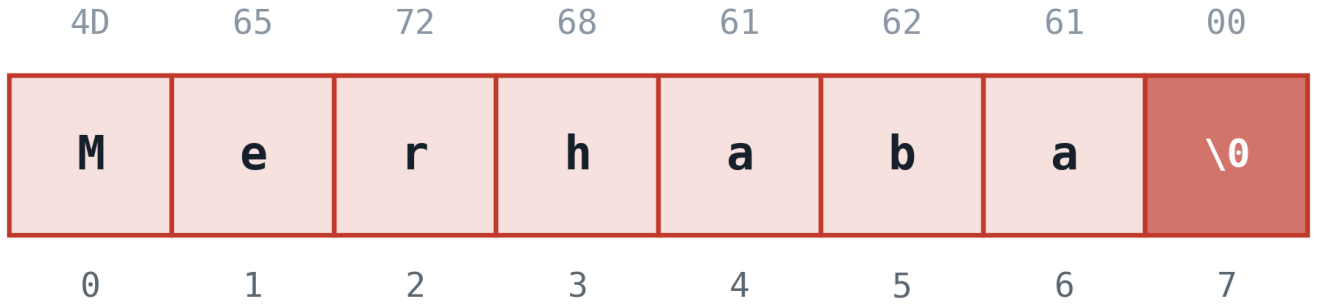
01

String nedir?

'\0' ile biten char dizisi · boyut + 1 · sizeof ↔ strlen · dizi mi değişmez mi

String: '\0' ile biten bir bayt şeridi

char ad[] = "Merhaba"; → 8 bayt (7 harf + sonlandırıcı)



üstte bayt değeri (onaltılık), altta indis; '\0' (00) string'in SONUNU imler

strlen = 7 ('\0' hariç) • sizeof = 8 ('\0' dahil)

- Her string `'\0'` (değeri 0) ile **biter**: SONU imler
- `printf("%s")` '\0' bulana dek ilerler
- "boyut + 1" kuralı: n karakter → en az n + 1 bayt
- `sizeof` = tipin baytı (8); `strlen` = '\0'a kadar karakter (7)

Dizi mi, değişmez mi? (K01)

H13_K01_string_degismez.c · satır 19–28

```c
int main(void)
{
    char s[] = "abc";          /* diziye kopya: 4 bayt, değiştirilebilir */
    const char *p = "abc";           /* değişmezi gösterir: salt okunur say */

    s[0] = 'X';                      /* GÜVENLİ: dizinin kendi baytı */
    printf("s değişti: %s\n", s);

    /* p[0] = 'X';  <- TANIMSIZ DAVRANIŞ: değişmezi değiştirme (HA6) */
    printf("p (değişmez): %s\n", p);
```

GERÇEK ÇIKTI

```
s değişti: Xbc
p (değişmez): abc
sizeof s = 4 (dizinin baytları, '\0' dahil)
sizeof p = 8 (yalnızca işaretçinin boyutu)
```

- `char s[]`: diziye kopya, değiştirilebilir (4 bayt)
- `char *p`: salt okunur değişmez; `p[0]='X'` tanımsız (HA6)
- `sizeof s` = 4 (dizi), `sizeof p` = 8 (işaretçi)

TAHMİN ET · DİZİ BOYUTU VE UZUNLUK

`char s[] = "abc";` → `sizeof s` ve `strlen(s)`?

quiz/H13_Q04_sizeof_strlen.c · satır 8–13

```c
int main(void)
{
    char s[] = "abc";
    printf("%zu %zu\n", sizeof s, strlen(s));
    return 0;
}
```

A) 4 3

B) 3 3

C) 3 4

D) 4 4

02

Güvenli girdi

scanf'in tuzakları · fgets ve '\n' · akışlar (stdin/stdout/stderr)

ÖRNEK 2 ★ · H13_02

scanf tehlikeli, fgets güvenli (Örnek 2)

H13_02_fgets_ad_soyad.c · satır 22–25

```
22      if (fgets(ad, sizeof ad, stdin) == NULL) {   /* EOF ya da hata */
23          printf("Girdi okunamadı.\n");
24          return 1;
25      }
```

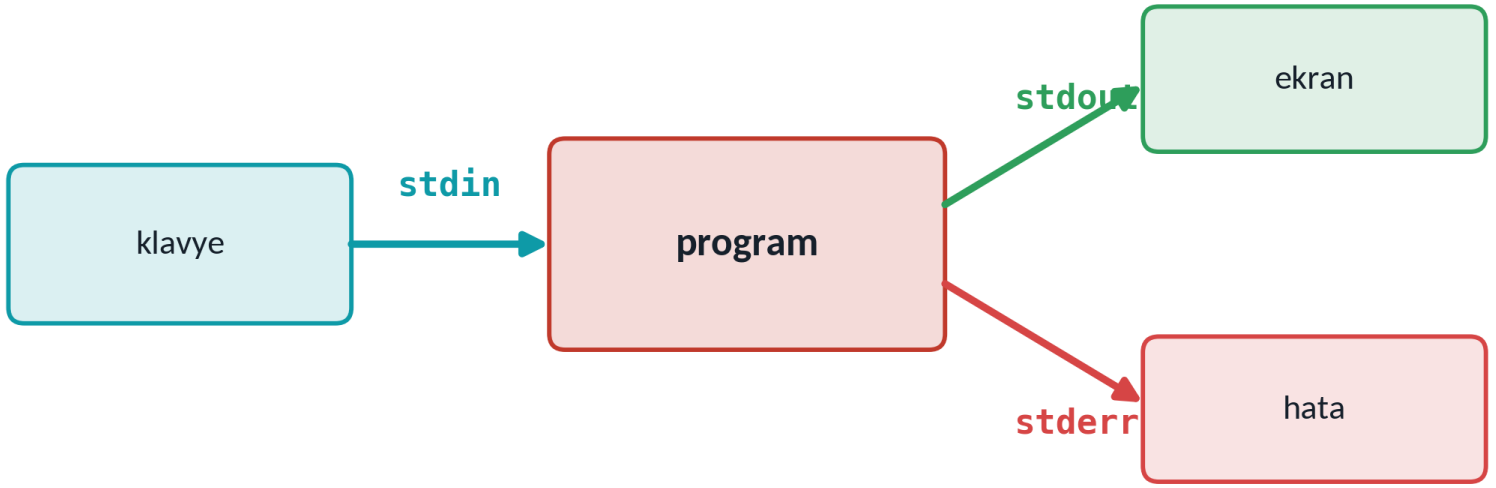
GERÇEK ÇIKTI (GİRDİ: AYŞE NUR)

```
Ad soyad girin: Ayşe Nur
Okunan: [Ayşe Nur] (9 bayt)
```

- `scanf("%s")`: boşlukta durur, genişliksiz TAŞAR
- `fgets`: en çok `sizeof-1` bayt, '\0' garantili → taşmaz
- Sondaki '\n'i `strcspn` ile sil (HA4)

Akışlar: stdin, stdout, stderr (H14'e köprü)

Akışlar (stream): program üç standart akışla konuşur (H14'e köprü)



fgets stdin'den okur; printf stdout'a, fprintf(stderr,...) hata iletisi yazar

- **stdin**: standart girdi (klavye); `fgets`/`scanf` okur
- **stdout**: standart çıktı (ekran); `printf` yazar
- **stderr**: standart hata; `fprintf(stderr,...)` yazar
- `./prog < girdi.txt`: aynı akış dosyadan beslenir
- H14'te `FILE *` ile dosyalara genişleteceğiz

TAHMİN ET · SATIR SONU TUZAĞI

Girdi "evet" ise `fgets` sonrası `strcmp(cevap, "evet") == 0` sonucu?

quiz/H13_Q12_fgets_newline.c · satır 9–17

```c
int main(void)
{
    char cevap[16];
    if (fgets(cevap, sizeof cevap, stdin) == NULL) {
        return 1;
    }
    printf("%s\n", strcmp(cevap, "evet") == 0 ? "eşit" : "eşit değil");
    return 0;
}
```

A. eşit (başarılı)

B. eşit değil ('\n' kaldı)

C. program çöker

D. derlenmez

03

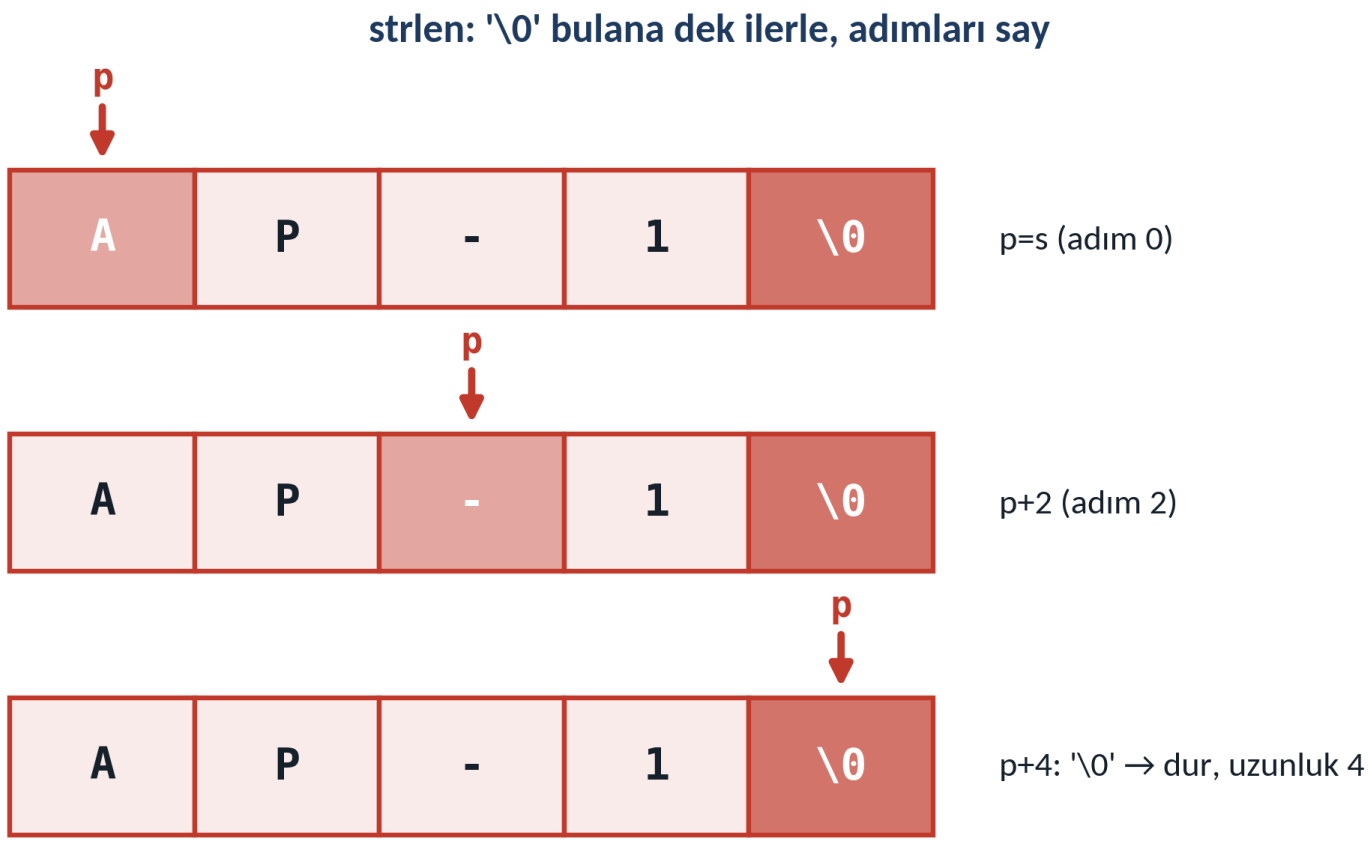
string.h ve kendi gerçeklememiz

strlen, strcpy, strcmp, strchr, strstr, strtok · tuzaklar

string.h: en sık kullanılan fonksiyonlar

Fonksiyon	Ne yapar?	Dikkat
`strlen(s)`	'\0'a kadar karakter sayısı	'\0' hariç
`strcpy / strncpy`	kopyalar / en çok n bayt	strncpy '\0' koymayabilir
`strcat / strncat`	sona ekler / en çok n bayt	hedef yeterli olmalı
`strcmp(a, b)`	sözlük sırasına göre karşılaştırır	yalnızca İŞARET anlamlı
`strchr / strstr`	karakterin / alt string'in konumu	bulamazsa NULL
`strtok(s, ayr)`	ayraçlardan parçalar	s'yi DEĞİŞTİRİR; durum tutar

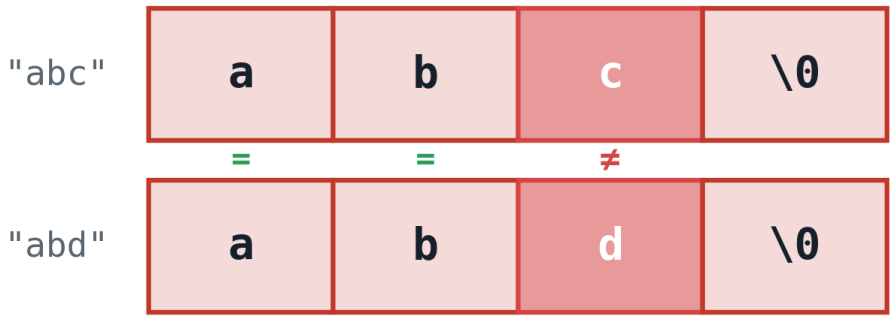
Kendi strlen'imiz: '\0' bulana dek ilerle (Örnek 5)



- Sır: **'\0' bulana dek ilerle, adımları say**
- İndisle: `while (s[n] != '\0') n++;`
- İşaretçiyle: `while (*p != '\0') p++;` → `p - s` (H12)
- Boş string: yalnızca '\0', uzunluk 0

strcmp: ilk farklı bayt işareti belirler (Örnek 8)

strcmp("abc", "abd"): bayt bayt karşılaştır, ilk farkta dur



'c'(99) − 'd'(100) = −1 → NEGATİF → "abc" sözlükte önce gelir

yalnızca dönüşün İŞARETİ anlamlıdır (negatif / 0 / pozitif)

- Baytları karşılaştır, ilk farkta dur
- Farkın (unsigned char) **İŞARETİ** anlamlı
- Negatif → ilk metin sözlükte önce
- Kesin sayı taşınabilir değil

İki tuzak: strncpy ve strtok

Boyut sınırlı fonksiyonların da incelikleri vardır (K06, K07).

strncpy

- kaynak n'den uzunsa '\0' KOYMAYABİLİR
- sonuç '\0'sız → sonraki strlen taşar
- çözüm: son baytı elle '\0' yap

strtok

- özgün diziyi DEĞİŞTİRİR (ayraca '\0')
- durum tutar: ilk çağrı dizi, sonrakiler NULL
- değişmezde: tanımsız davranış (burada çöktü, HA5)

TAHMİN ET · KARŞILAŞTIRMANIN İŞARETİ

`strcmp("elma", "armut")` dönüşü ne anlama gelir?

quiz/H13_Q06_strcmp_isaret.c · satır 8–13

```
 8  int main(void)
 9  {
10      int r = strcmp("elma", "armut");
11      printf("%s\n", r > 0 ? "elma sonra gelir" : "elma önce gelir");
12      return 0;
13  }
```

A Negatif: "elma" önce gelir

B Pozitif ama anlamsızdır

C Pozitif: "elma" sonra gelir

D Sıfır: eşittirler

04

Karakter algoritmaları

harf dönüşümü · ters çevirme · palindrom · Sezar · kelime sayma · frekans

ÖRNEK 3 ★ · H13_03

Harf dönüşümü: ASCII aritmetiği ve ctype (Örnek 3)

H13_03_harf_donusturme.c · satır 25–32

```
25      for (size_t i = 0; metin[i] != '\0'; i++) {
26          char c = metin[i];
27          /* yalnızca ASCII küçük harfleri FARK kadar yukarı kaydır */
28          ascii[i] = (c >= 'a' && c <= 'z') ? (char)(c - FARK) : c;
29          ctyp[i] = (char)toupper((unsigned char)c);   /* unsigned char ZORUNLU */
30      }
31      ascii[sizeof metin - 1] = '\0';
32      ctyp[sizeof metin - 1] = '\0';
```

GERÇEK ÇIKTI

```
Girdi  : AP-1 dersi
ASCII  : AP-1 DERSI
toupper: AP-1 DERSI
'a'-'A' = 32; 'z' kodu 122, 'Z' kodu 90
```

- ASCII: `'a' - 'A' = 32` (H3)
- ctype: `toupper((unsigned char)c)`
- İki yöntem ASCII harfler için aynı

String'i yerinde ters çevirme (Örnek 6)

H13_06_ters_cevir.c · satır 18–31

```
18      size_t i = 0;
19      size_t j = strlen(s);
20      if (j == 0) {
21          return;                     /* boş string: yapılacak bir şey yok */
22      }
23      j--;                            /* son karakterin indisi ('\0'dan önce) */
24      while (i < j) {
25          char gecici = s[i];         /* üç atamayla takas (H3) */
26          s[i] = s[j];
27          s[j] = gecici;
28          i++;
29          j--;
30      }
31  }
```

GERÇEK ÇIKTI

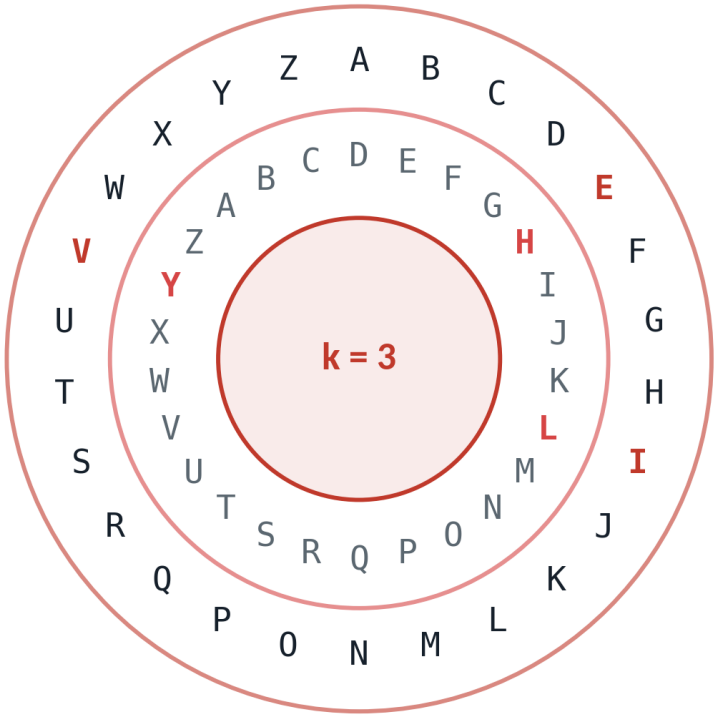
```
"merhaba" -> "abahrem"
"AP-1 2026" -> "6202 1-PA"
"x" -> "x"
```

- H10 iki uç tekniği: i baştan, j sondan
- s[i] ile s[j] takas, i < j oldukça
- BAYT bazlı: yalnızca ASCII doğru çevrilir

ÖRNEK 9 ★★ · AMİRAL ÖRNEK

Sezar çarkı: modüler aritmetikle kaydırma (Örnek 9)

Sezar çarkı (k = 3): dış halka açık, iç halka şifreli



V→Y E→H N→Q I→L · çözme: (26 − k) ile · anahtar uzayı yalnızca 26

Her harf k kadar kayar (mod 26); çözme (26 − k) ile · anahtar uzayı yalnızca 26: zayıf şifre

ÖRNEK 9 ★★ · H13_09

Sezar: şifreleme, çözme ve kaba kuvvet

H13_09_sezar.c · satır 22–31

```
22      k = ((k % 26) + 26) % 26;        /* negatif anahtar da çalışsın */
23      for (size_t i = 0; s[i] != '\0'; i++) {
24          unsigned char c = (unsigned char)s[i];
25          if (c >= 'A' && c <= 'Z') {
26              s[i] = (char)('A' + (c - 'A' + k) % 26);
27          } else if (c >= 'a' && c <= 'z') {
28              s[i] = (char)('a' + (c - 'a' + k) % 26);
29          }                            /* harf değilse dokunma */
30      }
31  }
```

GERÇEK ÇIKTI (K=3 VE KABA KUVVETİN BİR KISMI)

```
Açık metin : VENI VIDI VICI
k = 3 şifre : YHQL YLGL YLFL
Çözülmüş   : VENI VIDI VICI
...
  k= 2: WFOJ WJEJ WJDJ
  k= 3: VENI VIDI VICI
```

TAHMİN ET · SEZAR KAYDIRMA

"ABC" k = 2 ile Sezar şifrelenirse sonuç?

quiz/H13_Q09_sezar.c · satır 6–15

```c
int main(void)
{
    char s[] = "ABC";
    int k = 2;
    for (int i = 0; s[i] != '\0'; i++) {
        s[i] = (char)('A' + (s[i] - 'A' + k) % 26);
    }
    printf("%s\n", s);
    return 0;
}
```

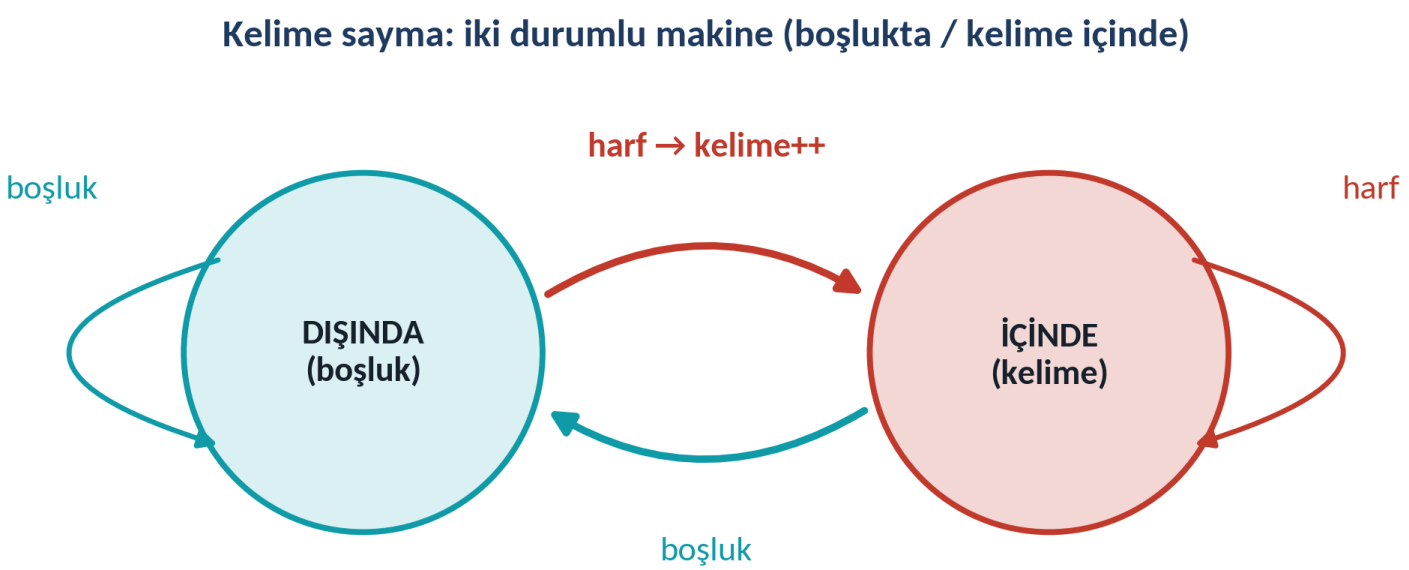
A CDE

B ABC

C DEF

D YZA

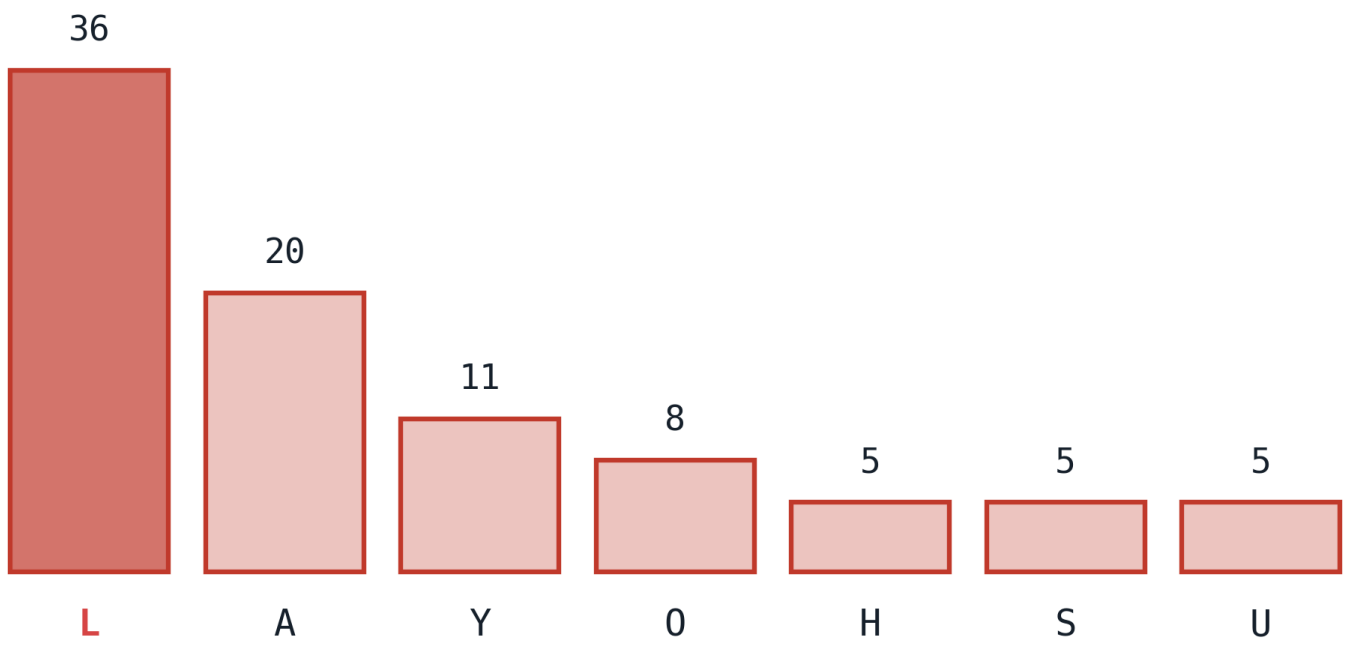
Kelime sayma: durum makinesi (Örnek 13)



Yalnızca 'dışından içeri' geçişte sayaç artar (yeni kelime başlangıcı)

- İki durum: **kelime içinde** / **dışında** (boşlukta)
- Yalnızca dıştan içe GEÇİŞTE sayaç artar
- `stdin`'den `EOF`'a kadar `getchar` ile oku (akış)
- Gerçek: `bir iki üç\ndört beş` → 24 bayt (20 karakter), 5 kelime, 2 satır

Frekans analiziyle Sezar kırma (Örnek 14)



- H10 frekans dizisi: `sayac[c - 'A']`
- İngilizcede en sık harf **'E'**
- En sık şifreli harf → 'E' varsayımı
- `anahtar = (en_sik - 'E' + 26) % 26`
- Gerçek: en sık L → anahtar 7, metin çözülür

ctype.h ve unsigned char kuralı (K09)

H13_K09_ctype.c · satır 16–27

```
16      char c = 'A';
17      unsigned char uc = (unsigned char)c;   /* ctype'a vermeden önce ZORUNLU */
18
19      printf("'%c' harf mi? %s; rakam mı? %s; boşluk mu? %s\n", c,
20             isalpha(uc) ? "evet" : "hayır",
21             isdigit(uc) ? "evet" : "hayır",
22             isspace(uc) ? "evet" : "hayır");
23      printf("isupper('%c') = %s; tolower -> '%c'\n", c,
24             isupper(uc) ? "evet" : "hayır", tolower(uc));
25      printf("toupper('g') -> '%c'; isdigit('7') -> %s\n",
26             toupper((unsigned char)'g'),
27             isdigit((unsigned char)'7') ? "evet" : "hayır");
```

GERÇEK ÇIKTI

```
'A' harf mi? evet; rakam mı? hayır; boşluk mu? hayır
isupper('A') = evet; tolower -> 'a'
toupper('g') -> 'G'; isdigit('7') -> evet
(char)0xC3 = -61 (işaretliyse negatif); unsigned char = 195
```

- `isalpha`, `isdigit`, `isspace`, `toupper` …
- Argüman önce `(unsigned char)` **olmalı** (STR37-C)
- `char` işaretliyse 0x80+ bayt negatif → tanımsız

String → sayı: atoi hatayı bildiremez, strtol bildirir

H13_K10_strtol.c · satır 16–23

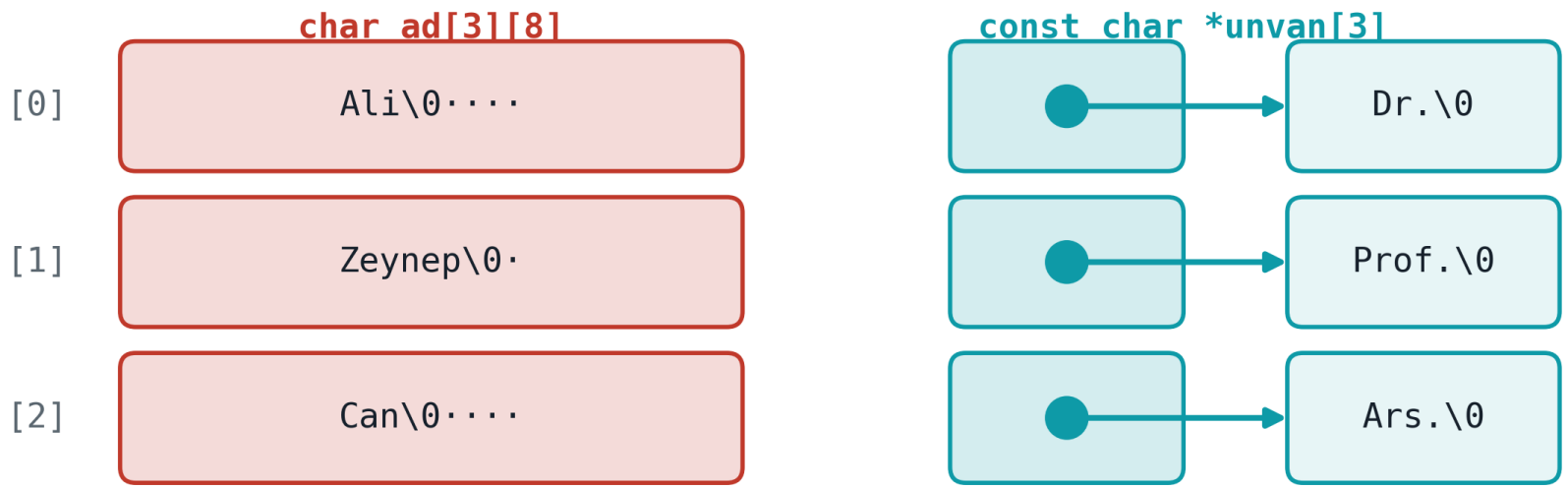
```
16      char *son;
17      errno = 0;
18      long deger = strtol(metin, &son, taban);
19      if (son == metin) {
20          printf("\"%s\": hiç rakam yok\n", metin);
21      } else if (errno == ERANGE) {
22          printf("\"%s\": taşma (ERANGE)\n", metin);
23      } else if (*son != '\0') {
```

STRTOL: BEŞ DURUM

```
"2026": 2026 (taban 10, tamamı sayı)
"12abc": sayı 12, kalan "abc" (tamamı sayı değil)
"": hiç rakam yok
"99999999999999999999": taşma (ERANGE)
"0x1F": 31 (taban 16, tamamı sayı)
```

- `atoi("abc")` = 0: hata ile 0'ı ayırt edemez
- `strtol`: `endptr` (nerede durdu), `errno`/`ERANGE` (taşma)
- H6 basamak biriktirme: `sonuc*10 + (c-'0')`

String dizileri: bitişik blok vs işaretçi dizisi (Örnek 11)



tek bitişik blok: 3×8 = 24 bayt
(kullanılmayan baytlar da yer kaplar)

3 işaretçi + ayrı salt okunur
değişmezler (bitişik değil)

- `char ad[N][M]`: tek bitişik blok; satırlar değiştirilebilir
- `const char *unvan[]`: işaretçi dizisi; ayrı değişmezler
- En uzun isim: `strlen`; arama: `strcmp` doğrusal tarama
- Gerçek: en uzun “Abdullah” (8); “Zeynep” 3. sırada

Komut satırı argümanları: argc ve argv (Örnek 15)

H13_15_argv_hesap_makinesi.c · satır 34–40

```
34  int main(int argc, char *argv[])
35  {
36      printf("argc = %d; argv[0] (program adı) = \"%s\"\n", argc, argv[0]);
37      printf("argv[argc] == NULL mı? %s\n",
38             argv[argc] == NULL ? "evet" : "hayır");
39
40      if (argc != 4) {
```

GERÇEK ÇALIŞTIRMALAR

```
$ ./hesap 12 x 5
argc = 4; argv[0] (program adı) = "./hesap"
argv[argc] == NULL mı? evet
12 x 5 = 60

$ ./hesap 100 - 58
argc = 4; argv[0] (program adı) = "./hesap"
argv[argc] == NULL mı? evet
100 - 58 = 42
```

- `int main(int argc, char *argv[])`
- `argc`: argüman sayısı (program adı dahil)
- `argv[0]` program adı; `argv[argc] == NULL`
- Sayılar `strtol` ile denetlenir

TAHMİN ET · ARGÜMAN SAYIMI

`./q10 bir iki` komutuyla çalıştırılınca `argc` kaçtır?

quiz/H13_Q10_argv.c · satır 7–12

```
 7  int main(int argc, char *argv[])
 8  {
 9      (void)argv;
10      printf("argc = %d\n", argc);
11      return 0;
12  }
```

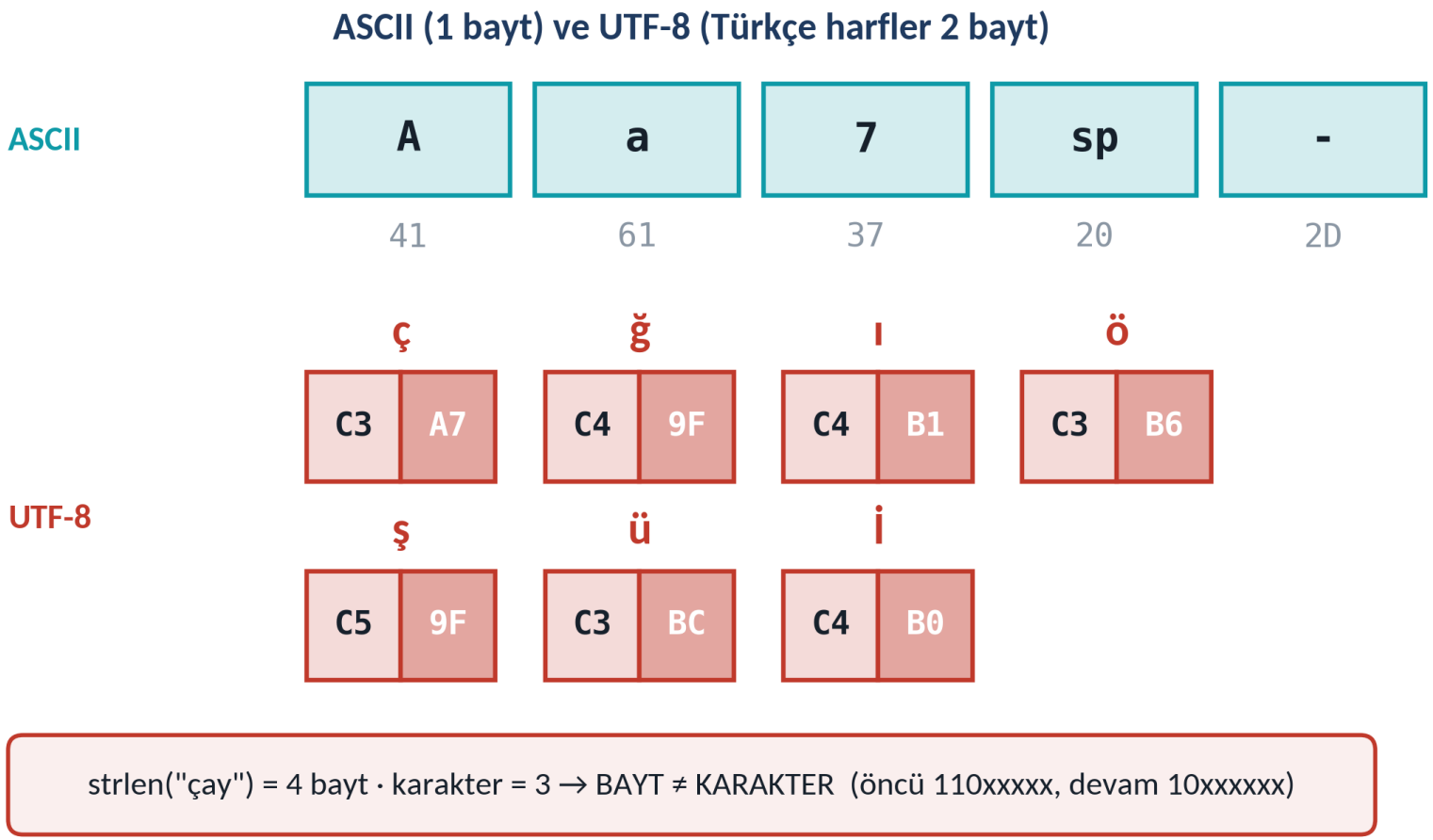
A 2

B 3

C 1

D 4

UTF-8: bayt ≠ karakter (Örnek 12)



- ASCII: her karakter 1 bayt; Türkçe harfler UTF-8'de **2 bayt**
- `strlen("çay")` = 4 bayt, karakter = 3
- GCC varsayılan kaynak/yürütme kodlaması UTF-8
- Devam baytı 10xxxxxx sayılmaz → karakter sayısı

TAHMİN ET · “TÜRKÇE I SORUNU”

`toupper((unsigned char)'i')` hangi karakteri döndürür?

quiz/H13_Q08_toupper_turkce.c · satır 8–12

```c
int main(void)
{
    printf("%c\n", toupper((unsigned char)'i'));
    return 0;
}
```

A `'İ'`

B `'ı'`

C `'I'`

D `'i'`

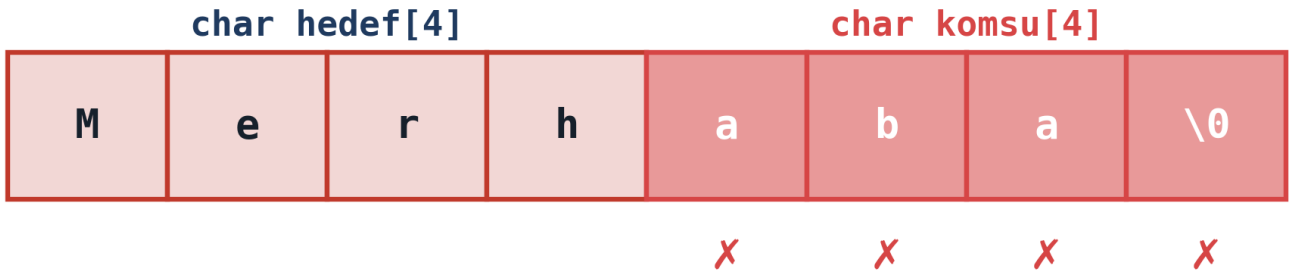
05

Güvenli kodlama ve tampon taşması

strcpy taşması · yığın çerçevesi · ASan ve kanarya · güvenli sürüm

strcpy taşması: komşu bellek ezilir

strcpy(hedef, "Merhaba") — hedef 4 baytlık: komşu bellek EZİLİR

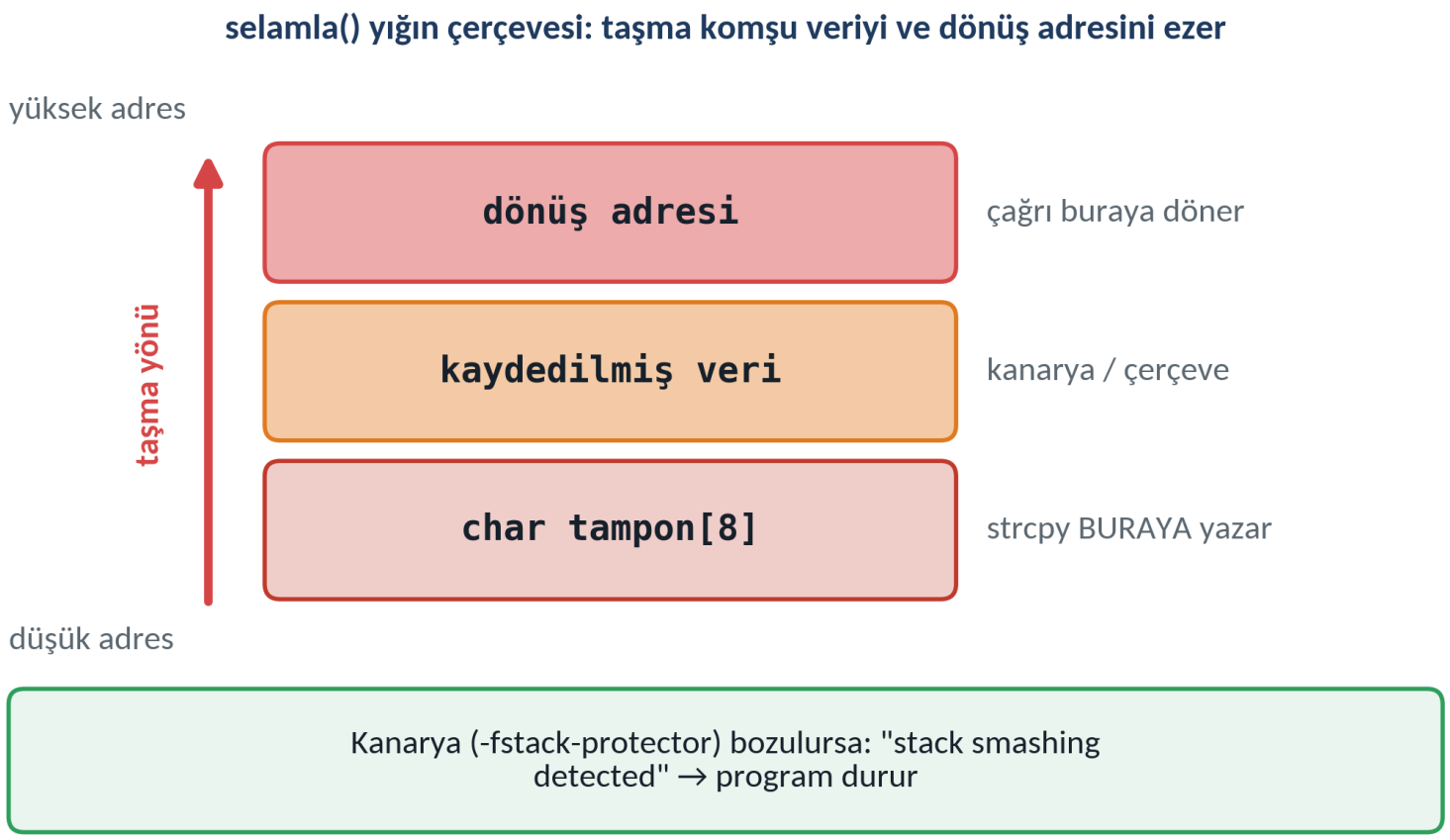


Kaynak "Merhaba" = 8 bayt; yalnızca 4 bayt ayrıldı → 4 bayt komşuya taşar

Güvenli: strncpy/snprintf + boyut denetimi (STR31-C)

8 baytlık kaynak 4 baytlık hedefe kopyalanınca 4 bayt komşuya taşar · güvenli: strncpy/snprintf + boyut (STR31-C)

Yığın çerçevesinde taşma: dönüş adresi tehlikede



Taşan baytlar komşu veriyi ve dönüş adresini ezer; kanarya (-fstack-protector) bunu saptar

Örnek 16: tampon taşması — kanarya ve ASan (gerçek)

H13_16_tampon_tasmasi.c · satır 27–32

```
27  void selamla(const char *isim)
28  {
29      char tampon[8];                 /* yalnızca 8 bayt: 7 karakter + '\0' */
30      strcpy(tampon, isim);           /* HATA: sınır denetimi yok, TAŞABİLİR */
31      printf("Merhaba, %s!\n", tampon);
32  }
```

ASAN'SIZ: KANARYA

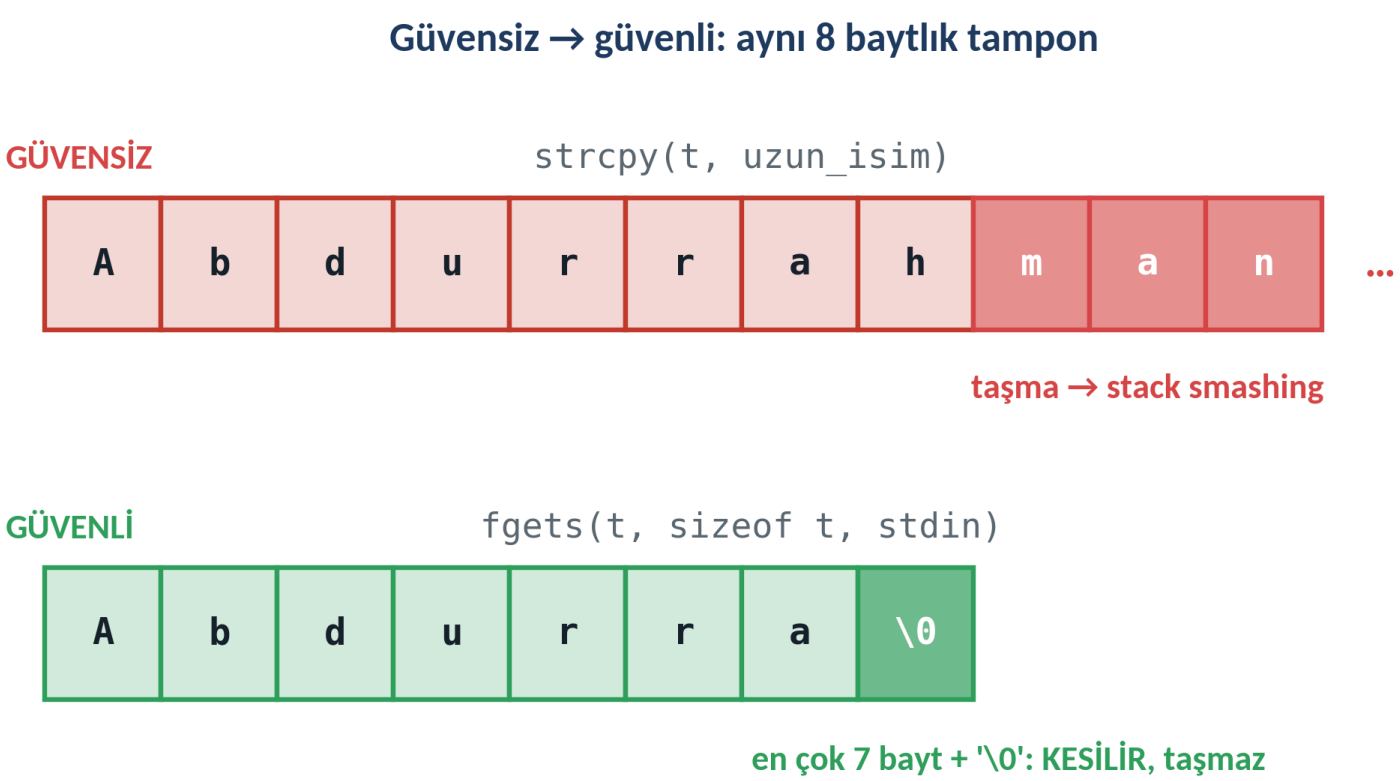
```
$ ./tampon_tasmasi   # uzun ad
*** stack smashing detected ***: terminated
bash: line 1:  9683 Aborted
    ./tampon_tasmasi
çıkış durumu: 134
```

ADDRESSSANİTİZER: STACK-BUFFER-OVERFLOW

```
ERROR: AddressSanitizer: stack-buffer-overflow

WRITE of size 19
    [32, 40) 'tampon' (line 29) <== Memory
    access at offset 40 overflows this
    variable
SUMMARY: AddressSanitizer:
    stack-buffer-overflow
    ../../../../src/libsanitizer/asan/asan_int
    erceptors.cpp:563 in strcpy
```

Güvenli sürüm: fgets ve snprintf (Örnek 16b)



- `fgets(t, sizeof t, stdin)`: en çok boyut-1 bayt, taşmaz
- Uzun ad güvenle **KESİLİR** (veri kaybı, bellek güvenli)
- `snprintf`: dönüş ≥ boyut → kesilme saptanır
- Tam 7 karakter (“Mehmet7”) kesilmez: `'\n'` yoksa bir sonraki karakter okunur
- ASan altında rapor yok: güvenli

06

Hata Avcısı

altı sık yanılgı ve düzeltmeleri

Hata Avcısı: hata | belirti | çözüm

Hata	Belirti	Çözüm
`char kod[5]="HELLO"`	sınır dışı okuma (ASan)	`char kod[] (boyut+1)`
`girdi == dogru`	içerik aynıyken 'farklı'	`strcmp(...)==0`
`scanf("%s", ad)`	uzun girdide taşma	`scanf("%7s") / fgets`
`strcmp(cevap,"evet")`	'\n' yüzünden eşleşmez	`strcspn ile '\n' sil`
`strtok("a,b", ...)`	tanımsız; burada çöktü (139)	`char csv[]="..."`
`char *s="abc"; s[0]='X'`	tanımsız; burada çöktü (139)	`char s[]="abc"`

HA4: fgets'in bıraktığı '\n' (hatalı → düzeltilmiş)

Hatalı: '\n' silinmedi

hata_avcisi/H13_HA4_fgets_newline.c · satır 19–23

```c
19      if (fgets(cevap, sizeof cevap, stdin) == NULL) {
20          return 1;
21      }
22      /* HATA: cevap "evet\n", "evet" ile asla eşleşmez */
23      if (strcmp(cevap, "evet") == 0) {
```

Doğru: strcspn ile sil

hata_avcisi/H13_HA4_fgets_newline_duzeltilmis.c · satır 17–21

```c
17      if (fgets(cevap, sizeof cevap, stdin) == NULL) {
18          return 1;
19      }
20      cevap[strcspn(cevap, "\n")] = '\0';   /* DÜZELTME: sondaki '\n'i sil */
21      if (strcmp(cevap, "evet") == 0) {
```

“evet” girince: hatalı sürüm “Onaylanmadı”, düzeltilmiş sürüm “Onaylandı” der.

HA6: string değişmezine yazma (hatalı → düzeltilmiş)

Hatalı: değişmeze yazma

hata_avcisi/H13_HA6_degismez_yazma.c · satır 19–23

```
19      char *s = "abc";                /* değişmezi gösterir (salt okunur) */
20      printf("önce: %s\n", s);
21      s[0] = 'X';                     /* HATA: değişmeze yazma (tanımsız) */
22      printf("sonra: %s\n", s);
23      return 0;
```

Doğru: diziye kopya

hata_avcisi/H13_HA6_degismez_yazma_duzeltilmis.c · satır 13–17

```
13      char s[] = "abc";               /* diziye kopya: değiştirilebilir */
14      printf("önce: %s\n", s);
15      s[0] = 'X';                     /* GÜVENLİ: dizinin kendi baytı */
16      printf("sonra: %s\n", s);
17      return 0;
```

Hatalı sürüm tanımsız davranış: bu sistemde çöktü (Segmentation fault, 139); dizi sürümü güvenle “Xbc” der.

Mühendis Gözüyle: güvenli kodlama

Metin işleme, C güvenlik açıklarının en yoğun alanıdır; dört alışkanlık riski azaltır.

Boyutu bil, sınırla

- her tampon için boyut bilinir
- fgets/snprintf/strncpy + boyut
- gets asla (C11'de kaldırıldı)

'\0' ve değişmez

- her string '\0' ile biter
- değişmezler salt okunur (STR30-C)

Araçla doğrula

- ASan ve UBSan
- -fstack-protector (kanarya)
- valgrind (Linux/WSL)

SEI CERT C

- STR30/31/32/37-C
- ERR34-C (strtol)
- CWE-120 / CWE-121

Atölye etkinlikleri

Kullan–Değiştir–Yarat: üç atölye haftanın araçlarını birleştirir (çözümler 92_Degerlendirme).

1 · Metin çözümleme (wc)

- karakter/kelime/satır sayımı
- durum makinesi (Örnek 13)
- genişletme: en uzun satır, frekans

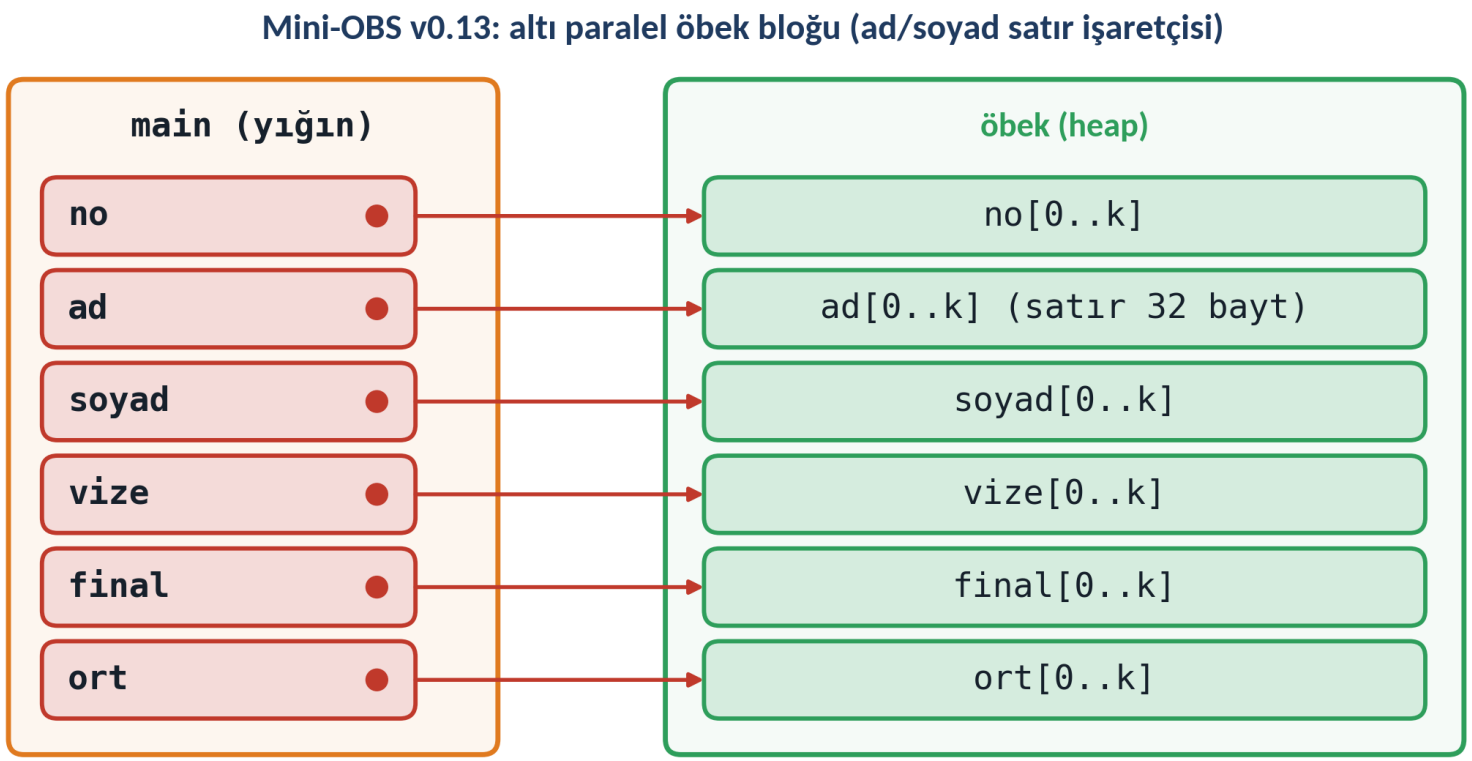
2 · Sezar + frekans kırma

- şifrele, kaba kuvvet + frekans analizi
- anahtar uzayı neden 26?
- sınıf yarışması

3 · Güvensiz → güvenli

- gets→fgets, strcpy→sınırlı, sprintf→snprintf
- her dönüşüm hangi CERT kuralı?
- ASan ile doğrula

Mini-OBS v0.13: ad–soyad, isimle arama, güvenli girdi



ad: char(*)[32] · notlar: double(*)[3] · kapasite dolunca altısı da realloc ile 2 katına

Altı paralel blok zahmeti → H14'te tek struct Ogrenci

- Ad/soyad öbekte **satır işaretçisiyle**: `char (*ad)[32]`
- `ad_normallestir`: boşluk temizleme + Türkçe büyük harf
- `isim_bul`: strcmp ile doğrusal tarama (tüm eşleşmeler)
- Güvenli girdi: `satir_oku`+`strtoll`/`strtod`; `scanf` kalktı
- Testler 79 → **108** doğrulama (29 metin testi)

AP-2 Köprüsü

strcmp → string sıralaması

İki metni sözlük sırasına göre karşılaştırmak, string dizilerini sıralamanın temeli; qsort'a karşılaştırma fonksiyonu.

strstr → desen eşleme

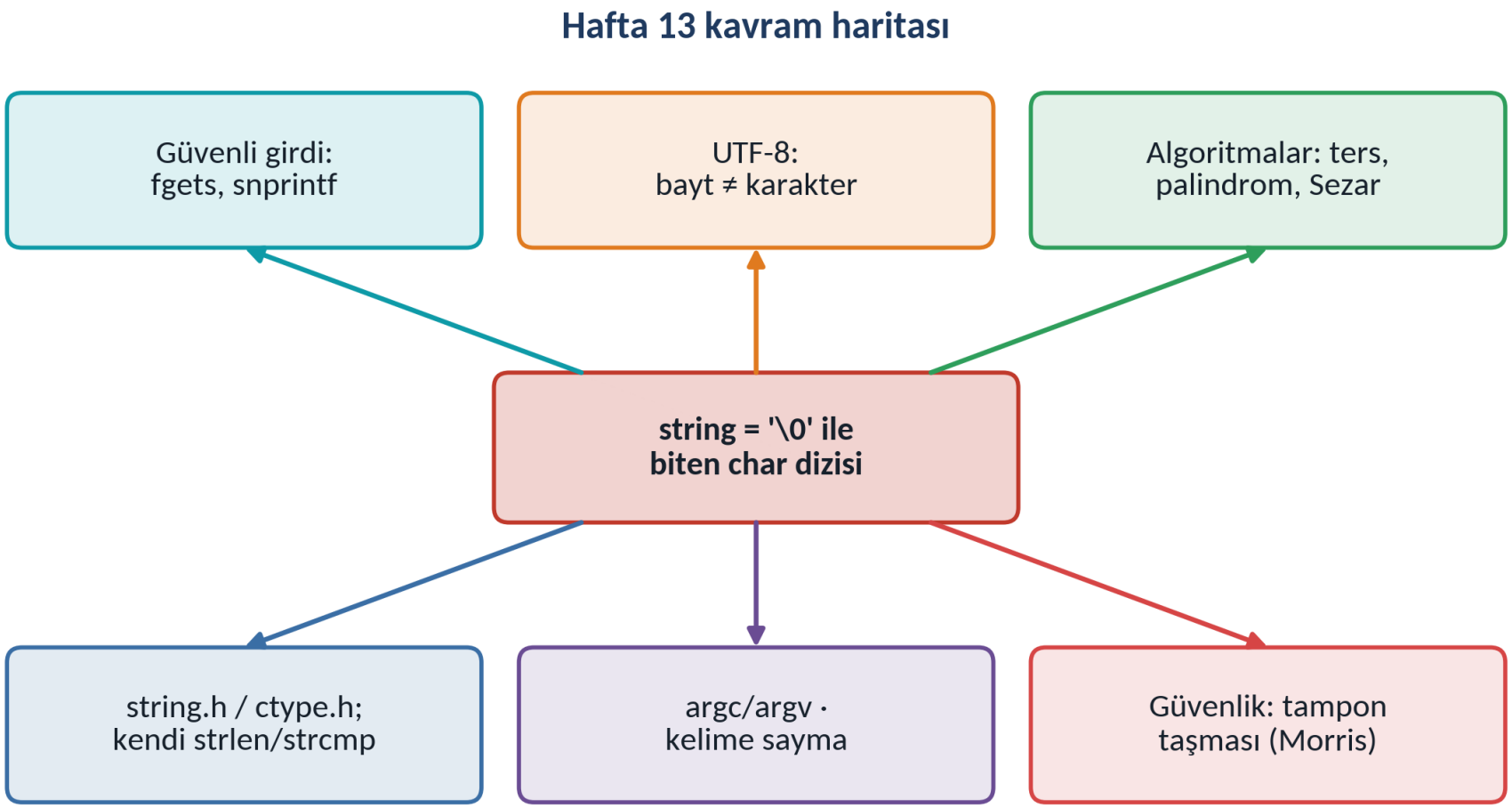
Alt string aramak, desen eşleme algoritmalarına (naif, Knuth–Morris–Pratt) giriş noktası.

Frekans → karma (hashing)

Karakterin kendisini indis yapmak, karma tablosu ve sayma sıralaması sezgisinin ilk adımı.

Bu haftanın yapı taşları AP-2'de biçimsel algoritma ailelerine dönüşür (burada yalnızca sezgi).

Özet: Hafta 13 kavram haritası



ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1 `char s[6] = "merhaba";` neden tehlikelidir? Bir cümleyle yazın.

2 `fgets` ile okunan bir satırı “evet” ile karşılaştırmadan önce hangi iki adım gerekir?

3 `strlen("gün")` kaçtır, neden? (kaynak UTF-8)

Haftaya Hazırlık ve Kaynaklar

Gelecek hafta (H14): Yapılar, Dosyalar ve Final Projesi

- Altı paralel blok yerine tek `struct Ogrenci` (ad + numara + notlar bir kayıtta)
- Veriyi diske yazma: `FILE *`, fopen/fclose, fprintf/fscanf (bu haftanın akış kavramının genişlemesi)
- Mini-OBS v1.0: kalıcı saklama ve bütünleştirici final projesi

Hazırlık ve kaynaklar

- Web crash course: string simülasyonları (bellek görselleştirici, Sezar makinesi, UTF-8 inceleyici, tampon taşması)
- Kernighan & Ritchie (1988) Böl. 5; King (2008) Böl. 13, 23; Seacord (2013); SEI CERT C (2016)
- `kodlar/` klasöründeki 16 örneği derleyip çalıştırın; Örnek 16'yı ASan ile deneyin